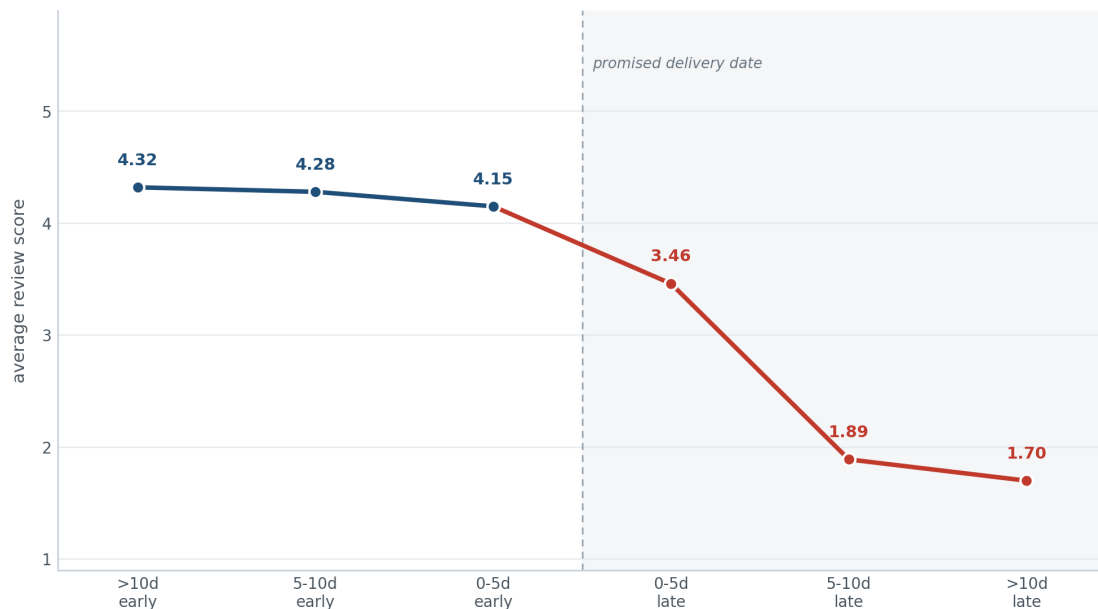

Olist Brazilian E-Commerce

An End-to-End SQL-First Analytics Project

DuckDB • SQL → Parquet → Power BI



Average review score by delivery delay bucket, 96,478 delivered orders.

Md. Ashiqur Rahman Khan (Ashik)

B.Sc. in Statistics, Mawlana Bhashani Science and Technology University

ashiqurrahmankhan04@gmail.com |

<https://github.com/Ashiq-r-khan/olist-ecommerce-analytics>

Dataset: Olist Brazilian E-Commerce Public Dataset, September 2016 to August 2018

August 2026

Summary

Olist is a Brazilian marketplace integrator. It gives independent sellers a shared checkout, a delivery partner network and a review system, and it takes a cut of the transactions that flow through it. In 2018 it published an anonymised dataset of the orders placed on the platform between September 2016 and August 2018.

This report analyses that dataset. There are 99,441 orders in it, of which 96,478 were delivered. Those delivered orders carry R\$15,419,773.75 of revenue, an average order value of R\$159.83, an on-time rate of 91.9% and a mean review score of 4.09.

The pipeline is built in DuckDB. All the cleaning, the dimensional model and the business logic sit in SQL, and Power BI is used only to display the results. Ten questions were answered in the SQL layer and published to a four-page dashboard.

Five results matter.

1. **Review score falls off a cliff at the promised date.** It barely moves across the whole early range, 4.32 for more than ten days early down to 4.15 for zero to five days early. Once the promise is missed it drops to 3.46, then 1.89, then 1.70. Customers are judging the promise, not the delivery time.
2. **The on-time rate comes from padding, not from speed.** Average delay is negative in all 27 states. Olist adds 8 to 19 days of buffer to the estimate shown at checkout. The size of that buffer predicts which states miss their dates. Delivery speed does not.
3. **Growth came from new customers only.** Monthly orders rose from about 750 to 6,500 over 24 months. Average order value stayed inside R\$145 to R\$175 the whole time.
4. **There is almost no repeat business.** Of 93,358 unique customers, 97.0% ordered once. Every 2017 cohort retains under 0.7% at every offset from month one to month six.
5. **Both sides of the marketplace are concentrated.** 133 sellers out of 2,970 produce half the revenue, and São Paulo alone is 37.4% of it.

The recommendation that follows from the first two results is narrow. Do not spend on median delivery speed, because the median order already sits in the flat part of the curve where speed changes nothing. Spend on the tail instead. The 4.27% of orders that arrive more than five days late average 1.89 and produce 74 to 79% one and two star reviews, and no other variable in the dataset moves review score by anything close to that much.

Contents

Summary	i
1 Introduction	1
1.1 Background	1
1.2 Objectives	1
1.3 Scope and conventions	2
2 Data	3
2.1 Source	3
2.2 Data quality problems	3
2.2.1 Geolocation is a point cloud	3
2.2.2 Reviews are not unique per order	4
2.2.3 Payments sit below order grain	4
2.2.4 customer_id is an order-scoped key	4
3 Method	6
3.1 Pipeline	6
3.2 Keeping the logic in SQL	6
3.3 Dimensional model	7
3.4 Tools	7
4 Findings	9
4.1 Revenue growth	9
4.2 Category performance	9
4.3 Revenue by state	10
4.4 Delivery performance by state	11
4.5 Delivery delay and review score	12
4.5.1 The correlations point the other way	12
4.6 Payment behaviour	13
4.7 Seller concentration	14
4.8 Customer retention	14
4.9 Review behaviour	15

4.9.1	Response speed	15
4.9.2	Who writes a comment	16
5	Dashboard	17
5.1	Executive overview	17
5.2	Geography	18
5.3	Delivery and satisfaction	19
5.4	Customers and sellers	20
5.5	Model design	20
6	Recommendations	22
6.1	Work on the delivery tail, not the median	22
6.2	Keep the buffer, but set it per state	22
6.3	Test whether retention is even possible before spending on it	22
6.4	Monitor the seller concentration	23
6.5	Do not act on the review response time result	23
7	Limitations	24
8	Conclusion	25
A	BI view layer	26
B	Selected SQL	27
C	Reproduction	29

List of Figures

5.1	Executive overview: headline KPIs, monthly revenue with year-on-year growth, the 2018 category treemap, and the order status distribution.	17
5.2	Geography: revenue by state, the freight against basket size scatter, and the concentration table.	18
5.3	Delivery and satisfaction: the delay bucket cliff, the buffer against late rate scatter, and promised against actual delivery by state.	19
5.4	Customers and sellers: cohort sizes, the seller Pareto curve, the retention matrix, and instalment behaviour.	20

List of Tables

2.1	Source files as loaded in Step 01. Names are shown without the <code>olist_</code> prefix and <code>_dataset</code> suffix carried by the first eight.	3
3.1	Pipeline steps	6
3.2	Mart layer	7
4.1	Top categories by revenue, 2018	10
4.2	State revenue concentration, delivered orders	11
4.3	Buffer size against late rate, two states	12
4.4	Review score by delivery delay bucket	12
4.5	Average order value by instalment count, selected tiers	13
4.6	Seller revenue concentration, 2,970 sellers with delivered revenue	14
4.7	Cohort retention, percentage of the cohort ordering again by month offset	15
A.1	Published views	26

1. Introduction

1.1 Background

Olist does not sell anything itself. It connects small and medium Brazilian merchants to the large marketplaces and supplies the parts each of them would otherwise have to build alone: one checkout, a logistics partner network, and a review system that runs after delivery. That makes it a platform rather than a retailer. It earns on the flow of transactions, and its main risks are concentration risks.

In November 2018 the company released an anonymised dataset covering roughly 100,000 orders placed between September 2016 and August 2018. Customer, seller and product identifiers are hashed, and seller names are replaced with the names of Game of Thrones houses. The timestamps, the geolocation, the payment structure, the freight values and the review scores are real.

The unusual part is the delivery lifecycle. Most public e-commerce datasets give an order date and nothing else. This one keeps five timestamps per order: purchase, approval, handover to the carrier, delivery to the customer, and the estimate that was shown at checkout. The last of those is what makes Chapter 4 possible, because it allows lateness to be measured against the promise instead of against the clock.

1.2 Objectives

The project has two objectives.

The analytical objective is to answer ten business questions: revenue growth, category performance, geographic concentration, delivery performance, review score, payment behaviour, seller concentration, retention, cohort behaviour and review response. Each question is stated, answered in SQL, and then interpreted. Where the obvious reading of a result is wrong, the report says so and explains why.

The architectural objective is to build the whole thing SQL-first. Every aggregation, every derived column, every bucket boundary and every window function is computed in the warehouse. Power BI reads views that are already finished and adds formatting. Section 3.2 gives the reasons for working this way.

1.3 Scope and conventions

Unless a table says otherwise, every figure in this report is computed on delivered orders only, 96,478 of 99,441. Cancelled, unavailable, shipped, invoiced, processing, approved and created orders are left out. They are not completed transactions, and including them would lower the average review score and raise the order count without adding revenue.

Revenue means item price plus freight, summed across the items in an order. All currency is Brazilian reais and no inflation adjustment has been applied. Timestamps are used as supplied.

Calendar year 2016 is in the data but is excluded from every year-on-year comparison. September 2016 has one order and December 2016 has one order. Dividing by those months produces numbers that look like growth rates and are not.

2. Data

2.1 Source

The dataset is nine CSV files in a normalised structure, joined on `order_id`, `customer_id`, `product_id`, `seller_id` and `zip_code_prefix`.

Table 2.1: Source files as loaded in Step 01. Names are shown without the `olist_` prefix and `_dataset` suffix carried by the first eight.

File	Rows	Grain and content
orders	99,441	One row per order, with status and five lifecycle timestamps
order_items	112,650	One row per item line, with price and freight
order_payments	103,886	One row per payment instrument per order
order_reviews	99,224	One row per review, with duplicates on some orders
customers	99,441	Order-scoped key plus the stable unique key
products	32,951	Category, dimensions, weight, photo count
sellers	3,095	Seller location
geolocation	1,000,163	Latitude and longitude points per zip prefix
product_category_name_translation	71	Portuguese to English category names

2.2 Data quality problems

Four problems had to be settled before any of the analysis was valid. All four were fixed in the SQL staging layer rather than later.

2.2.1 Geolocation is a point cloud

The geolocation file has a million rows because it stores every coordinate that was ever observed, not one per postcode. Joining it straight onto orders multiplies rows badly.

It was reduced to one centroid per zip prefix, using the median of latitude and longitude and a bounding box around Brazil to drop points that fall outside the country. That gives **19,010** usable centroids. The median was used rather than the mean because the outliers being filtered are extreme and one-sided, and a mean would pull the centroids towards them.

2.2.2 Reviews are not unique per order

Some orders carry more than one review row, from resubmissions and follow-ups. These were reduced to one review per order, keeping the most recent by creation date. A window function does this in one pass, without a self-join:

```
1 FROM raw.reviews
2 QUALIFY ROW_NUMBER() OVER (
3     PARTITION BY order_id
4     ORDER BY TRY_CAST(NULLIF(review_creation_date, '' ) AS TIMESTAMP) DESC
5             ,
6             TRY_CAST(NULLIF(review_answer_timestamp, '' ) AS TIMESTAMP)
7             DESC
8 ) = 1;
```

Listing 2.1: Review deduplication (Step 02)

The result is **98,673** reviews. Where two rows share a creation date, the answer timestamp breaks the tie, so the output is the same on every run.

2.2.3 Payments sit below order grain

An order can be paid with more than one instrument. The common case is a voucher topped up by a credit card, which appears as two rows sharing an `order_id`. Joining payments to orders without aggregating first counts that revenue twice.

Payments were therefore aggregated to order grain in the fact layer, keeping the total value, the maximum instalment count, the number of distinct payment types and a dominant payment type. Section 4.6 shows that vouchers split across rows 63.6% of the time, so this is not a small correction.

2.2.4 `customer_id` is an order-scoped key

This is the trap in the dataset. Olist issues a new `customer_id` for every order and keeps the real identity in `customer_unique_id`. If retention is keyed on `customer_id`, the repeat rate comes back as exactly 0.0000% across 96,478 identifiers.

All customer-level analysis in this report keys on `customer_unique_id`.

Why this one is dangerous

The wrong key and the right key tell the same story, which is that Olist has almost no repeat business. A 0% result would not have looked odd next to the correct 3%. Nothing downstream would have caught it either, because the number is impossible by construction rather than merely out of range. The only defence is reading the schema before writing the query.

3. Method

3.1 Pipeline

The pipeline runs in eight steps, in Google Colab, against a local DuckDB file. Power BI reads the output through Parquet.

Table 3.1: Pipeline steps

Step	What it does
01	Loads the nine CSVs into a <code>raw</code> schema as all-VARCHAR, so that loading cannot fail on an unexpected type and every cast is made explicitly later.
02	Staging. Typed casts, geolocation centroids, review deduplication, timestamp checks, and a data quality audit table.
03	Mart. Four fact tables and five dimensions.
04	SQL EDA. The ten questions, answered with window functions.
05	BI layer. Ten views in a <code>bi</code> schema, written out to Parquet.
06	Dashboard. DAX display measures and four report pages.
07	This report.
08	Publication to GitHub.

3.2 Keeping the logic in SQL

The rule for the project was that business logic stays in SQL. Power BI receives tables where the percentages, ranks, buckets and cumulative shares are already calculated, and DAX is used only for display measures such as the KPI cards.

There are three reasons for working this way.

The first is portability. A view written in SQL still works after a change of BI tool. DAX does not.

The second is testing. A SQL view can be run on its own, diffed against a previous version and kept under version control. Its output does not depend on which visual is calling it, so it can be checked without opening the report.

The third is correctness under filtering, and it is the one that actually changed the design. A percentage computed in the warehouse means one fixed thing. A percentage computed in DAX

recalculates against whatever slicer happens to be applied, quietly. Section 5.5 describes what that implied for the Power BI model.

3.3 Dimensional model

Step 03 builds a fact constellation rather than a single star schema. The data has four different grains and forcing them into one fact table would mean either duplicating rows or throwing information away.

Table 3.2: Mart layer

Table	Rows	Grain
fact_orders	99,441	One order. Lifecycle timestamps, delivery days, delay days
fact_order_items	112,650	One item line. Price, freight, seller, product
fact_payments	99,440	One order. Payments aggregated up from instrument grain
fact_reviews	98,673	One order, after deduplication
dim_customer	99,441	Both keys kept, plus city and state
dim_seller	3,095	Seller location. 2,970 of these have delivered revenue
dim_product	32,951	English category and physical attributes
dim_date	daily	Calendar spine with no gaps
dim_geo	19,010	One centroid per zip prefix

The derived measures are calculated once, in the fact layer, and used everywhere after that. `delivery_days` is `delivery` minus `purchase`. `delay_days` is `delivery` minus the estimate given at checkout, so a negative value means the order arrived early. The sign is worth stating because it reads backwards from the column name, and because every state in the dataset has a negative average.

3.4 Tools

DuckDB was chosen for three reasons. It runs in-process, so there is no server to set up. It reads CSV and writes Parquet without any extra library. And it supports the SQL this project depends on, including `QUALIFY`, ordered-set aggregates and `MODE`, which SQLite does not.

Google Colab supplies the compute. Power BI Desktop reads the Parquet files through a folder parameter, so no absolute path is stored inside the report.

Two rules that cost time to learn

DuckDB allows only one connection to a database file at a time, and a database file must never be written directly to a mounted cloud drive. The pipeline works on a local copy and closes the connection before copying the file back. Breaking either rule produces an error that looks like file corruption when it is really contention. Appendix [C](#) repeats both.

4. Findings

4.1 Revenue growth

Revenue grew from R\$127,000 in January 2017 to R\$1.15M in November 2017, then settled into a band of R\$1.0M to R\$1.13M a month through August 2018. November 2017 is Black Friday. It is 53.6% above October 2017, it is the largest month in the dataset, and 2018 never matched it even with higher baseline volume.

What did not change is the more useful part. Across the full 24 months, average order value stayed inside R\$145 to R\$175 while monthly orders went from roughly 750 to 6,500. All of the growth came from acquiring customers. Nothing in the data suggests Olist ever got an existing customer to spend more, and Section 4.8 explains why. There were hardly any existing customers.

Year-on-year growth slows steadily through 2018: January +745.5%, March +170.5%, May +99.1%, July +81.5%, August +52.6%. This is the base effect wearing off as the 2017 comparison months get bigger, not a decline. The drop in the final month is truncation. Data collection stops in August 2018.

4.2 Category performance

The top ten categories are stable in membership but not in order. `bed_bath_table` led 2017 at R\$580,949 and fell to third by 2018, passed by `health_beauty` (R\$866,810, up 60.2%) and `watches_gifts` (R\$749,738, up 46.5%). Only two categories changed. `auto` came in at eighth and `garden_tools` dropped out.

Table 4.1: Top categories by revenue, 2018

#	Category	Revenue (R\$)
1	health_beauty	866,810
2	watches_gifts	749,738
3	bed_bath_table	643,650
4	sports_leisure	604,390
5	computers_accessories	583,990
6	housewares	481,660
7	furniture_decor	470,020
8	auto	399,010
9	baby	291,740
10	cool_stuff	260,160

Rank is the least interesting column here. `watches_gifts` turns R\$219 to R\$247 per order on 3,418 orders. `bed_bath_table` turns R\$131 to R\$133 per order on 4,844. The revenue is similar and the businesses are not. One sells margin, the other sells traffic. A recommendation aimed at the top ten as a single group would be wrong for half of it, because whatever works on a R\$240 watch is not what works on a R\$132 duvet cover.

4.3 Revenue by state

Revenue is heavily concentrated. São Paulo on its own is 37.4%. SP, RJ and MG together are 62.5%. Seven states out of 27 reach 80.9%, and the other twenty share less than a fifth between them.

Table 4.2: State revenue concentration, delivered orders

State	Revenue (R\$)	% rev.	Cum. %	AOV (R\$)	Avg freight (R\$)
SP	5,768,518	37.41	37.4	142.45	17.33
RJ	2,055,402	13.33	50.7	166.43	23.95
MG	1,818,892	11.80	62.5	160.20	23.46
RS	861,279	5.59	68.1	161.17	24.81
PR	781,709	5.07	73.2	158.79	23.49
SC	595,128	3.86	77.1	167.83	24.85
BA	591,138	3.83	80.9	181.55	29.96
DF	346,123	2.24	83.1	166.41	23.86
GO	334,212	2.17	85.3	170.78	26.25
ES	317,658	2.06	87.4	159.23	24.57
PE	308,972	2.00	89.4	193.96	35.83

Freight runs in the opposite direction to demand. São Paulo pays the lowest average freight in the country and orders the most. The remote northern and northeastern states pay up to 2.85 times as much and order the least. Average order value inverts the same way: SP has the lowest AOV at R\$142.46 and Paraíba has the highest at R\$266.61.

The likely mechanism is basket batching. Where freight is a large share of the order cost, small orders stop making sense, so customers wait and buy several things at once to spread the shipping over more items. That produces exactly what the table shows, which is high freight with high AOV and low order frequency. It also feeds itself, because low order frequency prevents the delivery density that would bring freight down. Population alone does not explain São Paulo's share. This loop does.

4.4 Delivery performance by state

Median delivery time runs from 7.21 days in São Paulo to 25.88 days in Amazonas, a spread of 3.6 times on the same platform. At the 90th percentile the spread is wider, 15.77 days in SP against 39.45 days in Alagoas.

The structural result is in the delay column. `avg_delay_days` is negative in all 27 states, from -8.03 to -19.40 . The checkout estimate is padded by 8 to 19 days everywhere. São Paulo promises 19.14 days and delivers in 7.21. Amazonas promises 45.28 and delivers in 25.88. The 91.9% on-time rate is therefore produced by careful promising rather than fast logistics.

Buffer size, not speed, predicts which states fail.

Table 4.3: Buffer size against late rate, two states

State	Median delivery	Avg promised	Buffer	% late
RO (best late rate)	17.58	38.77	21.19	2.88
AL (worst late rate)	22.33	32.58	10.25	23.93

Rondônia is slower than average and has the best on-time record in the country. Alagoas is about five days slower at the median but misses its date eight times as often, because its 90th percentile of 39.45 days runs past its own 32.58-day promise. A state fails when its tail is longer than its buffer, whatever its median looks like. The operational question is therefore not where deliveries are slow. It is where variance is larger than the buffer.

4.5 Delivery delay and review score

Lateness is the strongest single driver of review score in the dataset. The shape of the effect is a cliff, not a slope.

Table 4.4: Review score by delivery delay bucket

Delay bucket	Avg review score
1. More than 10 days early	4.32
2. 5 to 10 days early	4.28
3. 0 to 5 days early	4.15
4. 0 to 5 days late	3.46
5. 5 to 10 days late	1.89
6. More than 10 days late	1.70

Across the entire early range the score moves 0.17 points over ten days. Arriving ten days early buys almost nothing over arriving one day early. Crossing zero is a different story. The share of one and two star reviews goes from 11.07% in the last early bucket to 73.81% in the five to ten days late bucket, which is 6.7 times higher. On-time orders average 4.29 and late orders average 2.57, a gap of 1.72 points.

4.5.1 The correlations point the other way

Pearson's r is -0.267 for `delay_days` and -0.334 for `delivery_days`. Raw duration correlates more strongly with review score than lateness does, which contradicts everything

above if it is taken at face value.

It should not be. Pearson's r measures linear association, and the delay effect is flat across the whole early range and then vertical. That is the shape linear correlation is least able to detect. Duration degrades smoothly instead, so r describes it well. The bigger coefficient belongs to the weaker result. The bucket table is measuring the relationship and the correlation is measuring how linear that relationship happens to be.

Note on method

A correlation coefficient is the wrong summary when a threshold effect is present, and it will understate the effect it is being asked to describe. Reporting r next to the bucket table and explaining the difference is more useful than reporting either one alone. Delay and duration are also correlated with each other, so neither coefficient isolates a causal effect.

4.6 Payment behaviour

Credit card is 75.5% of delivered orders, or 72,825. Boleto, the Brazilian bank slip, is 19.9% at 19,191. Voucher is 3.1% and debit is 1.5%. Boleto is single payment by construction, with an average instalment count of exactly 1.00 and a split rate of zero.

Table 4.5: Average order value by instalment count, selected tiers

Instalments	Avg order value (R\$)	Note
1×	100.62	
3×	143.21	
6×	210.36	
7×	187.47	1,556 orders, below its neighbours
9×	198.80	615 orders, below its neighbours
10×	414.12	5,135 orders

Basket size tracks instalment count almost monotonically, from R\$100.62 at one instalment to R\$414.12 at ten. The two exceptions are the interesting part. Seven and nine instalment plans sit below their neighbours in both value and volume, while eight and ten carry high volume and high value together. Round-numbered plans are the ones merchants promote, and the odd-numbered tiers are what is left over. The financing tier is therefore partly a merchandising choice, which makes it something the platform can act on.

Credit card carries the highest average order value at R\$166.18 against R\$144.33 for boleto. The gap between mean and median is wide for every method, though. For credit card it is R\$166.18

against a median of R\$109.41. Order values are right-skewed throughout this dataset and the mean sits above the typical basket everywhere.

Vouchers split across multiple payment rows 63.6% of the time. They rarely cover a whole order and mostly work as partial credit topped up by something else. This is the structure that would have inflated revenue if payments had not been aggregated to order grain first.

4.7 Seller concentration

Table 4.6: Seller revenue concentration, 2,970 sellers with delivered revenue

Threshold	Sellers	% of sellers	% of revenue
Top 15 sellers	15	0.51	17.3
50% of revenue	133	4.5	50.0
80% of revenue	551	18.6	80.0
90% of revenue	913	30.7	90.0

The distribution is a little more extreme than the usual 80/20 rule, since 18.6% of sellers produce 80% of revenue. Of the 3,095 registered sellers, 125 never completed a sale at all.

The top of the distribution is not one kind of business, in the same way the top categories were not. Rank four turned R\$230,797 on 348 orders, which is R\$663 an order. Rank three turned R\$231,220 on 1,772 orders, which is R\$130 an order. The revenue contribution is identical and the operations behind it are not, so seller support cannot treat the top of the list as one segment.

The business implication is exposure. Losing 133 accounts would halve platform GMV, and 133 accounts fit on one screen. For a marketplace whose main claim is breadth of supply, that is a dependency worth naming.

4.8 Customer retention

Of 93,358 unique customers with delivered orders, 90,557 ordered exactly once. That is 97.0%. Only 2,801 ever came back. Mean orders per customer is 1.033 and the highest any single customer reached is 15.

Repeat customers are worth 1.87 times more, R\$308.53 in lifetime value against a R\$165.17 average. Retention is not worthless here. There is simply almost none of it.

The cohort matrix makes the point more clearly than the averages do. Every 2017 cohort retains under 0.7% at every offset from month one to month six. The January 2017 cohort had 717

customers and returned two of them in month one, which is 0.28%, and three in month six.

Table 4.7: Cohort retention, percentage of the cohort ordering again by month offset

Cohort	M0	M1	M2	M3	M4	M5	M6
Jan 2017	100	0.28	0.28	0.14	0.42	0.14	0.42
Feb 2017	100	0.18	0.31	0.12	0.43	0.12	0.25
Mar 2017	100	0.44	0.36	0.40	0.36	0.16	0.16
Apr 2017	100	0.62	0.22	0.18	0.27	0.27	0.36
May 2017	100	0.46	0.46	0.29	0.29	0.32	0.41
Jun 2017	100	0.49	0.40	0.43	0.30	0.40	0.36
Jul 2017	100	0.53	0.35	0.24	0.29	0.21	0.32

There is no decay curve because there is nothing to decay from. Retention starts near zero and stays there. The movement between 0.2% and 0.6% across offsets is noise and should not be read as monthly variation in loyalty.

The rows are worth reading even though the columns are empty. Cohort sizes climb from 717 in January 2017 to 4,057 in August 2017, which is 5.7 times in seven months. The matrix shows an acquisition engine, not a retention problem.

This fits how the platform works. Olist is a checkout and logistics layer behind thousands of independent sellers, on marketplaces the customer was already using. The customer forms a relationship with a product, a seller or the host marketplace. Olist itself may never register as a brand at all.

4.9 Review behaviour

4.9.1 Response speed

Reviews answered in under a day average 3.88, which is the lowest of any bucket. One to three day responses average 4.17, three to seven day 4.14, and seven day and over 4.02.

This is reverse causality and the report states that plainly rather than leaving it to be misread. Complaints get triaged first. A one star review is an escalation and gets attention the same day, while a five star review waits in the queue. Response speed is a consequence of the score, not a cause of it. Nothing here supports a recommendation to respond faster in order to raise ratings, and that is the most tempting wrong conclusion available in this dataset.

4.9.2 Who writes a comment

Comment rate falls sharply as satisfaction rises. 76.6% of one star reviews carry written text, against 35.9% of five star and 31.2% of four star. Angry customers write. Satisfied ones click and leave.

The score distribution is J-shaped rather than normal: 57.8% five star, 19.3% four, 8.2% three, 3.2% two and 11.5% one. One star is 3.6 times more common than two star. Customers use the ends of the scale and mostly ignore the middle.

Both facts matter for any later work on `review_comment_message`. The text is not a representative sample of customer opinion. It is closer to a complaint corpus. Sentiment analysis run over it would describe the customers who complain, not the customers, and would report a much unhappier business than a 4.09 average score does.

5. Dashboard

The Power BI report presents these results on four pages. Every number on them comes from a SQL view, and no measure in the report recalculates a percentage.

5.1 Executive overview

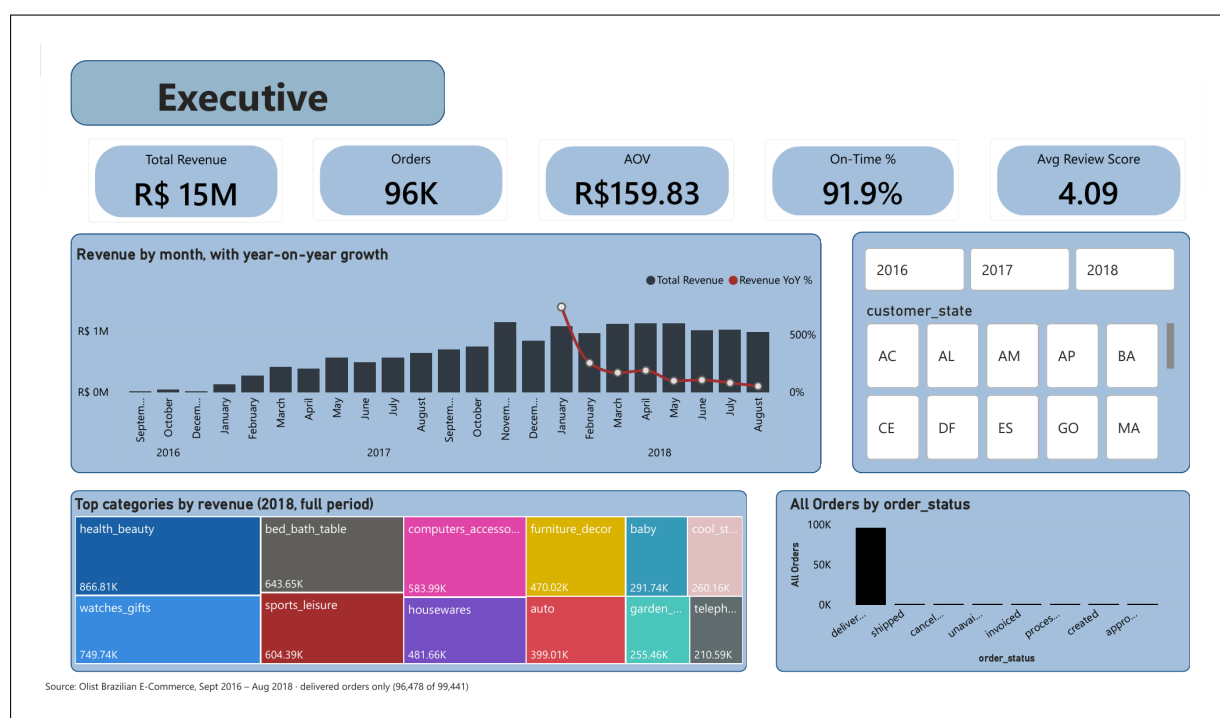


Figure 5.1: Executive overview: headline KPIs, monthly revenue with year-on-year growth, the 2018 category treemap, and the order status distribution.

Five KPI cards carry the headline numbers. The revenue column chart is paired with a year-on-year growth line on a secondary axis. That growth measure is guarded so that months with fewer than twenty prior-year orders return blank, which removes the 4,296,935% figure produced by dividing December 2017 by a December 2016 that held one order.

The category treemap uses a top ten plus other grouping computed in SQL, rather than a revenue cutoff. A treemap needs a predictable number of blocks to stay readable, and a fixed cutoff gives a different count every year while quietly dropping the tail.

5.2 Geography

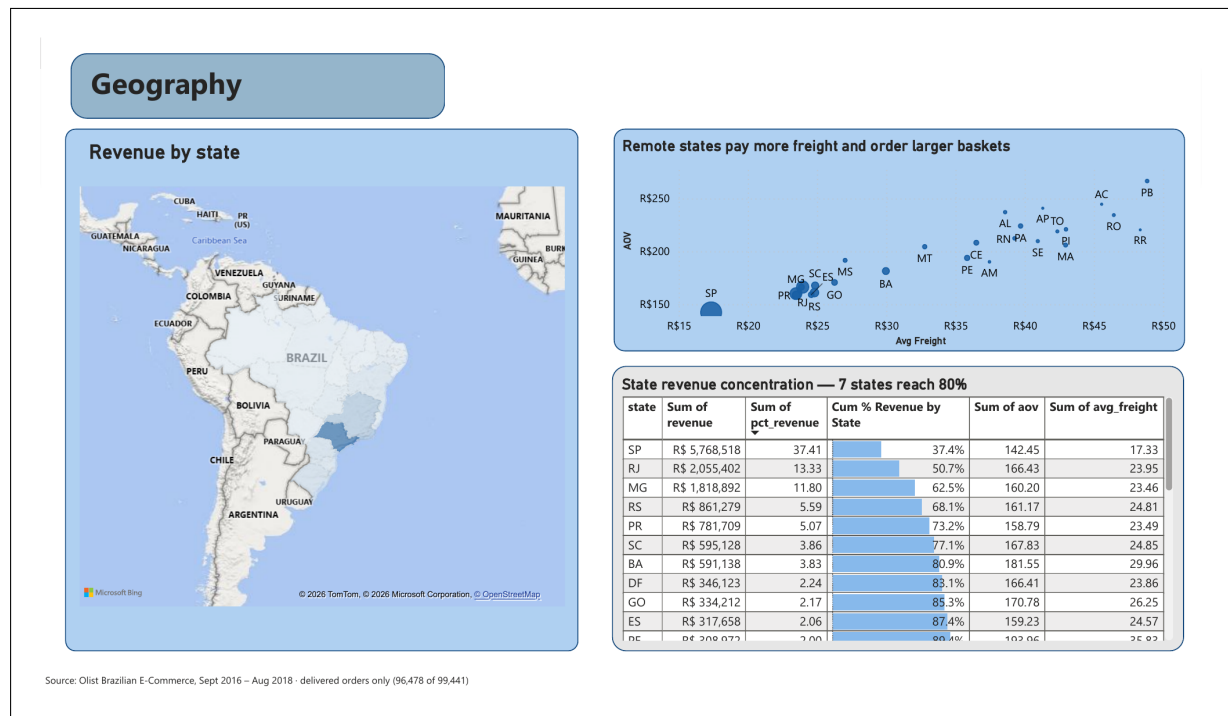


Figure 5.2: Geography: revenue by state, the freight against basket size scatter, and the concentration table.

The scatter is the useful visual on this page. Average freight on the horizontal axis, average order value on the vertical, one bubble per state sized by revenue. São Paulo sits on its own at the bottom left with the cheapest freight, the smallest baskets and the largest bubble, and the northern states cluster at the top right. The table next to it carries cumulative revenue share, with the 80% line falling at the seventh state.

5.3 Delivery and satisfaction

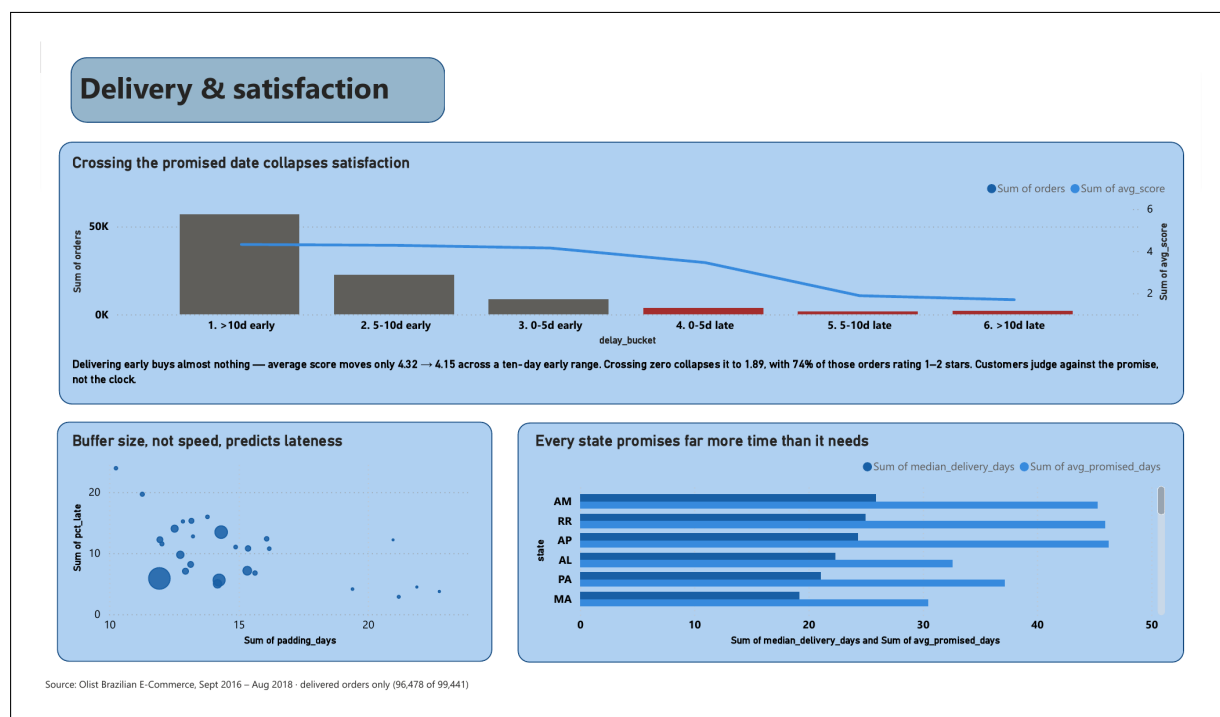


Figure 5.3: Delivery and satisfaction: the delay bucket cliff, the buffer against late rate scatter, and promised against actual delivery by state.

The combination chart is the most important visual in the project. Order volume as columns and average review score as a line, across the six delay buckets. The shape carries the argument on its own, flat across the three early buckets and then almost vertical after zero.

The buckets are sorted by an integer sort key rather than alphabetically. Without it the categorical axis puts “0-5d late” before “5-10d early” and the chart stops meaning anything.

The paired bar chart on the right shows promised against median actual delivery by state. The gap between the two bars is the buffer, and it is visible in every state.

5.4 Customers and sellers

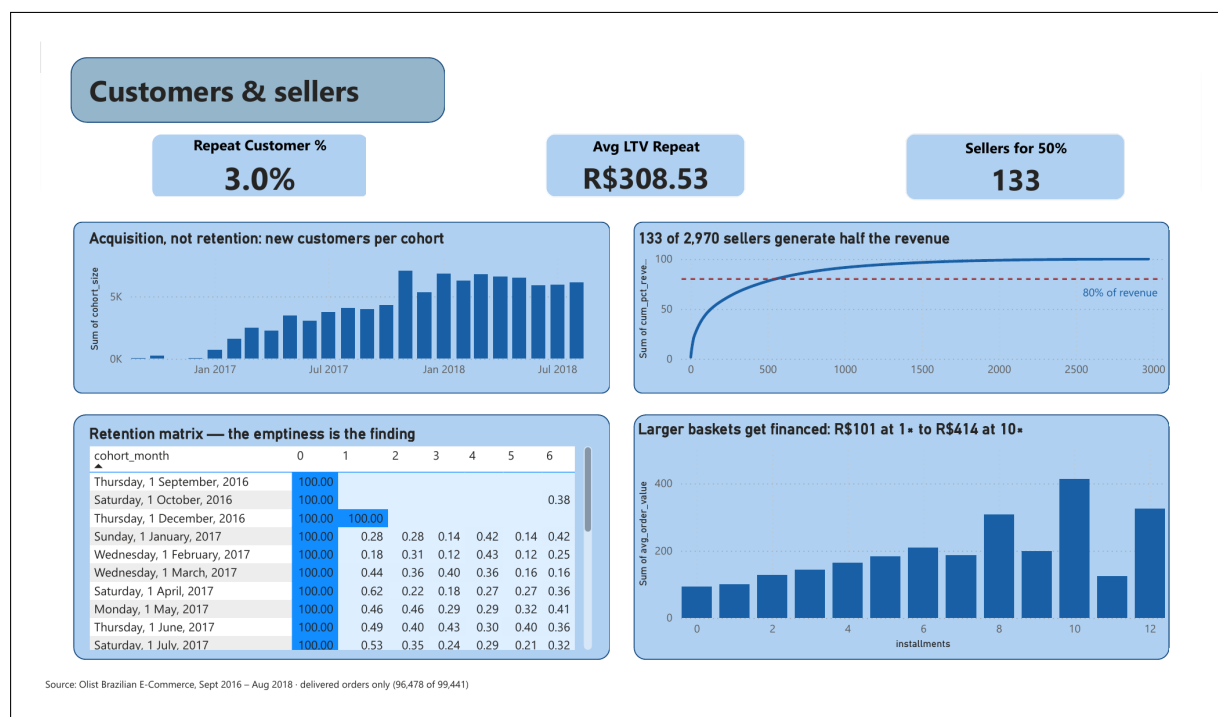


Figure 5.4: Customers and sellers: cohort sizes, the seller Pareto curve, the retention matrix, and instalment behaviour.

The Pareto curve climbs almost vertically before flattening, with a reference line at 80% of revenue. The retention matrix beside it is on the page because it is empty. A conditional formatting heat map with nothing to colour states the finding better than a summary statistic would.

5.5 Model design

The Power BI model has exactly two relationships, both single direction from fact to dimension. `vw_orders_enriched` joins `vw_dim_date`, and `vw_sales_daily` joins `vw_dim_date`. Every other view is disconnected on purpose: the delay review buckets, the seller Pareto, the cohort matrix, category performance, payment performance and delivery performance.

This is a trade-off and it costs something real. Those views carry percentages, cumulative shares and ranks that were computed in SQL. If a slicer could re-filter them, the percentages would stop summing correctly and a Pareto position calculated over a subset would still be labelled as marketplace-wide. Disconnecting them means every figure means what its title says. The price is that slicers only reach the pages built on `vw_orders_enriched`, so the cohort and seller

pages are static.

Grain is the second constraint. Both category and seller sit below order grain, in the `fact_order_items` table, so those visuals cannot respond to an order-level slicer without a bridge table or a re-grained fact.

For a dashboard whose job is to communicate a fixed set of findings, correctness at a fixed grain was worth more than interactivity that reports the wrong number quietly.

6. Recommendations

These follow from the findings and are ordered by how much evidence supports them.

6.1 Work on the delivery tail, not the median

Section 4.5 shows that the median order already sits in the flat part of the curve, where more speed changes the score by 0.17 points over a ten-day range. The 4.27% of orders that land more than five days late average 1.89 and produce 74 to 79% one and two star reviews. Taking a day off the median would move almost nothing. Removing half of that tail would move the average review score by a visible amount.

In practice this means routing and carrier changes should be judged on their effect on the 90th and 95th percentiles of delivery time by state, not on the mean. Alagoas is the obvious place to start, since its p90 of 39.45 days already runs past its own 32.58-day promise.

6.2 Keep the buffer, but set it per state

The padding strategy is defensible. It is what holds 91.9% of orders on the good side of the cliff, and building a buffer is the right response to a threshold effect.

The change worth making is to set the buffer from the local delivery-time distribution instead of applying it uniformly. Rondônia pads by 21 days and is late 2.88% of the time, so it is promising time it does not need, and a long promise costs conversion at checkout. States whose buffer sits below their p90 are under-buffered and pay for it in one star reviews.

6.3 Test whether retention is even possible before spending on it

A 3% repeat rate is not a retention problem waiting for a loyalty programme. It is evidence that the platform has no direct relationship with the customer to retain.

The question to answer first is whether Olist can reach the customer at all. Does it hold contactable identity, does the host marketplace allow direct contact, and is the brand recognised at the point of purchase? If the answer is no, then retention spending is misdirected and acquisition efficiency is the only real lever. The cohort matrix rejects a strategy. It does not diagnose underperformance.

6.4 Monitor the seller concentration

The 133 accounts that produce half the revenue should be tracked as named risk, with churn indicators followed individually rather than in aggregate. Since that group contains both high-ticket and high-volume sellers with different failure modes, one combined seller health score would hide the difference that matters.

6.5 Do not act on the review response time result

Included for completeness. The relationship between fast responses and low scores is reverse causality and should be kept out of any operational target. A response-time SLA built on that correlation would produce a metric that improves as the business gets worse.

7. Limitations

Time window. The data stops in August 2018 and starts with a launch period too sparse to use. Movement in the final month is truncation. With one usable holiday season, no seasonality claim beyond the single Black Friday is supportable.

Anonymisation. Seller and customer identifiers are hashed and seller names are replaced, so no external data can be joined in. Category benchmarks, seller size and marketplace of origin are all unavailable, and product category is the only merchant-side attribute in the data.

Causality. Nothing here is a causal estimate. Delay and delivery duration are correlated with each other, and both are correlated with state, freight and category. The delay bucket table is a strong conditional description, not an identified effect. The review lag result is explicitly reverse-causal.

Revenue definition. Revenue is item price plus freight. Olist's take rate is not in the dataset, so every revenue figure here is gross merchandise value rather than platform income.

Known artefacts. A small number of orders show delivery times up to 209.63 days, clustered on 2017-09-19 and 2018-09-19. These are batch status closures rather than real deliveries. They stay in the fact table but are excluded from the delay analysis by a null filter. Eight orders carry a delivered status with null delivery timestamps and are excluded the same way.

Two figures that differ between layers. Average order value is R\$159.83 on delivered orders and R\$160.58 across all statuses with items. This report uses R\$159.83 throughout. The seller count reaching 50% of revenue is 133 in SQL and 132 in DAX, which is a rounding boundary in the stored cumulative percentage. This report uses 133.

8. Conclusion

The most useful result in this project is not the biggest number in it. Revenue concentration and seller concentration are real and worth managing, but they describe a marketplace behaving the way marketplaces behave.

The result that would change a decision is the shape of the satisfaction curve. Customers do not reward speed and they do not punish slowness. They punish a missed promise, and they punish it all at once rather than gradually.

That was only visible because the relationship was bucketed before it was summarised. Run the correlation first and trust it, and the answer points at delivery duration and produces a recommendation to ship faster. That is an expensive change aimed at the flat part of the curve. The bucket table cost nothing and pointed at the tail instead.

Two smaller things generalise past this dataset. The first is that a schema tells you what a column means, not just what type it is. The `customer_id` trap gives an answer that is wrong by construction and still looks reasonable, and no amount of checking further downstream would have caught it. The second is that computing business logic in the warehouse instead of the BI layer is a correctness decision before it is a portability one. A percentage that cannot be re-filtered is a percentage that cannot quietly turn into a different percentage.

A. BI view layer

The ten views published from the `bi` schema to Parquet and read by Power BI through a `FolderPath` parameter.

Table A.1: Published views

View	Contents
<code>vw_orders_enriched</code>	Order-grain spine: state, delay bucket, review score, status, revenue
<code>vw_sales_daily</code>	Daily revenue and order counts, keyed to the date dimension
<code>vw_category_perf</code>	Category revenue, AOV and rank by year
<code>vw_delivery_perf</code>	Median, p90, promised days, buffer and late rate by state, with centroids
<code>vw_delay_review</code>	Delay buckets with average score, order count and sort key
<code>vw_seller_pareto</code>	Seller rank with cumulative revenue share
<code>vw_customer_cohort</code>	Cohort month by month offset retention matrix
<code>vw_payment_perf</code>	Payment method and instalment tier behaviour
<code>vw_dim_date</code>	Calendar dimension passthrough
<code>vw_dim_geo_state</code>	State centroids for the map visual

B. Selected SQL

```

1 CREATE OR REPLACE TABLE staging.geolocation AS
2 WITH typed AS (
3     SELECT geolocation_zip_code_prefix      AS zip_prefix,
4            TRY_CAST(geolocation_lat AS DOUBLE) AS lat,
5            TRY_CAST(geolocation_lng AS DOUBLE) AS lng,
6            LOWER(TRIM(geolocation_city))     AS city,
7            UPPER(TRIM(geolocation_state))     AS state
8     FROM raw.geolocation
9 )
10 SELECT zip_prefix,
11        ROUND(MEDIAN(lat), 6) AS lat,
12        ROUND(MEDIAN(lng), 6) AS lng,
13        MODE(city)           AS city,
14        MODE(state)          AS state,
15        COUNT(*)             AS n_points
16 FROM typed
17 WHERE lat BETWEEN -33.75 AND 5.27
18        AND lng BETWEEN -73.99 AND -34.79
19 GROUP BY zip_prefix;

```

Listing B.1: Geolocation reduced to one centroid per zip prefix

```

1 CASE WHEN delay_days IS NULL THEN NULL
2     WHEN delay_days <= -10 THEN '1. >10d early'
3     WHEN delay_days <= -5 THEN '2. 5-10d early'
4     WHEN delay_days < 0 THEN '3. 0-5d early'
5     WHEN delay_days < 5 THEN '4. 0-5d late'
6     WHEN delay_days < 10 THEN '5. 5-10d late'
7     ELSE '6. >10d late'
8 END AS delay_bucket
9
10 CREATE OR REPLACE VIEW bi.vw_delay_review AS
11 SELECT delay_bucket,
12        CAST(SUBSTR(delay_bucket, 1, 1) AS INTEGER)
13        AS bucket_sort,
14        COUNT(*)
15        AS
16        orders,
17        ROUND(100.0 * COUNT(*) / SUM(COUNT(*) OVER ()), 2)
18        AS pct_of_orders,
19        ROUND(AVG(review_score), 2)
20        AS avg_score,

```



```

16      ROUND(100.0 * AVG(CASE WHEN review_score <= 2 THEN 1 ELSE 0 END),
17      2) AS pct_1_2_star,
18      ROUND(100.0 * AVG(CASE WHEN review_score = 5 THEN 1 ELSE 0 END),
19      2) AS pct_5_star
20 FROM bi.vw_orders_enriched
21 WHERE is_delivered AND delay_bucket IS NOT NULL AND review_score IS NOT
22      NULL
23 GROUP BY 1, 2;

```

Listing B.2: Delay bucketing in vw_orders_enriched, and the view behind the headline chart

```

1  CREATE OR REPLACE VIEW bi.vw_seller_pareto AS
2  WITH s AS (
3      SELECT i.seller_id,
4             sd.state AS seller_state,
5             SUM(i.item_revenue) AS revenue,
6             COUNT(DISTINCT i.order_id) AS orders
7  FROM mart.fact_order_items i
8  JOIN mart.fact_orders o USING (order_id)
9  LEFT JOIN mart.dim_seller sd ON sd.seller_id = i.seller_id
10 WHERE o.order_status = 'delivered'
11 GROUP BY 1, 2
12 )
13 SELECT seller_id, seller_state,
14        ROUND(revenue, 2) AS revenue,
15        orders,
16        ROUND(revenue / orders, 2) AS seller_aov,
17        ROW_NUMBER() OVER (ORDER BY revenue DESC) AS seller_rank,
18        ROUND(100.0 * SUM(revenue) OVER (ORDER BY revenue DESC ROWS
19        UNBOUNDED PRECEDING)
20        / SUM(revenue) OVER (), 2) AS cum_pct_revenue
21 FROM s;

```

Listing B.3: Seller Pareto with a cumulative window

C. Reproduction

1. Download the nine CSVs from Kaggle into the project `data/` folder.
2. Run `Step01` to `Step05` in order. Each notebook expects the DuckDB file on local disk. Do not point DuckDB at a mounted cloud drive, because it must not write through a FUSE mount. Work at `/content` and copy the file back afterwards.
3. Close the connection before copying the database file back. DuckDB allows one connection at a time, and a second attempt with different settings fails with a configuration mismatch rather than a lock message, which is misleading. With `jupysql`, close any open Python connection before loading the SQL extension.
4. Step 05 writes the Parquet files. Open the Power BI file, set the `FolderPath` parameter to that folder and refresh.

Environment: Google Colab, DuckDB in-process, `jupysql`, Power BI Desktop.

Repository:

<https://github.com/Ashiq-r-khan/olist-ecommerce-analytics>